

API REST Con Laravel 11 API REST con Laravel 11

Contents

1	API REST con Laravel	1
1.1	API en pocas palabras	1
1.1.1	¿Que es un EndPoint?	2
2	Introducción	2
3	API REST con Laravel 11	2
4	Cambios en Laravel 11	3
5	Configuración inicial con Laravel Sail	3
5.1	1. Crear un Proyecto con Laravel 11 usando Sail	3
6	Instalación de Sanctum y configuración de autenticación	3
7	CRUD para Posts	5
7.1	1. Crear modelo y controlador	5
7.2	2. Definir rutas de la API	6
7.3	3. Implementar métodos en <code>PostController.php</code>	6
8	Conclusión	7

1 API REST con Laravel

Vamos a ver las posibilidades de escribir una API REST con Laravel 10. Podemos hacer un CRUD como hemos hecho antes con el blog. Nos daremos cuenta de que, al final, nos va a quedar algo mucho más sencillo, puesto que no tendremos vistas ni plantillas, ni componentes... Sólo **rutas**, y **puntos de acceso** —=endpoints=—.

1.1 API en pocas palabras

Una **Interfaz de Programación de Aplicaciones** permite a dos sistemas comunicarse uno con otro. Proporciona el lenguaje y el contrato de como interactúan los dos sistemas. Cada API tiene documentación y especificaciones que determinan cómo se puede transferir la información.

Las API (WEB) generalmente se clasifican como SOAP o REST y ambas se usan para acceder a los servicios web.

SOAP se basa únicamente en XML para proporcionar servicios de mensajería, mientras que REST ofrece un método más ligero, utilizando direcciones URL en la mayoría de los casos para recibir o enviar información.

Vamos a crear APIs REST.

Para aprovechar al máximo las API, las empresas las usan para:

- Integrarse con API de terceros.
- Para uso interno.
- Para exponerla para uso externo.

1.1.1 ¿Que es un EndPoint?

Brevemente un endPoint es un extremo de un canal de comunicación. Cuando una API interactúa con otro sistema, los puntos de contacto de esta comunicación se consideran endpoints.

Cuando una API solicita información de una aplicación web o un servidor web, recibirá una respuesta. El lugar donde las API envían solicitudes y donde vive el recurso se denomina endpoint.

Para las API, un endpoint puede incluir una URL de un *servidor*. Cada endpoint es la ubicación desde la cual las API pueden acceder a los recursos que necesitan para llevar a cabo su función.

2 Introducción

Esta guía está diseñada para ayudarte a crear una API REST utilizando Laravel 11. Laravel 11 introduce algunos cambios importantes en su estructura por defecto, como la eliminación de ciertas configuraciones iniciales, rutas de API predefinidas, y soporte opcional para paquetes que deben ser instalados manualmente.

Esta actualización aborda los cambios y adapta el proyecto creado en Laravel 9 y 10 para ser compatible con Laravel 11.

3 API REST con Laravel 11

Esta guía está diseñada para ayudarte a crear una API REST utilizando Laravel 11. Laravel 11 introduce algunos cambios importantes en su estructura por defecto, como la eliminación de ciertas configuraciones iniciales, rutas de API predefinidas y soporte opcional para paquetes que deben ser instalados manualmente.

Esta actualización adapta el proyecto creado en Laravel 9 y 10 para que funcione con Laravel 11.

—

Esta guía está diseñada para ayudarte a crear una API REST utilizando Laravel 11. Laravel 11 introduce algunos cambios importantes en su estructura por defecto, como la eliminación de ciertas configuraciones iniciales, rutas de API predefinidas y soporte opcional para paquetes que deben ser instalados manualmente.

Esta actualización adapta el proyecto creado en Laravel 9 y 10 para que funcione con Laravel 11.

4 Cambios en Laravel 11

1. Eliminación de Rutas de API Predefinidas :: Laravel 11 ya no incluye rutas API preconfiguradas en el archivo `routes/api.php`. Estas deben agregarse manualmente si son necesarias.
2. Middleware de Autenticación de API :: El middleware `auth:api` ya no está configurado por defecto. Para utilizar autenticación de API con tokens personales, necesitas instalar manualmente Sanctum o Passport.
3. Paquetes Eliminados o Opcionales :: Algunos paquetes previamente incluidos, como Sanctum o Passport, deben ser instalados manualmente.
4. Instalación Simplificada de Características Opcionales :: Laravel 11 introduce el comando `php artisan install:api` para instalar configuraciones opcionales, como sanctum y `api.php`.

5 Configuración inicial con Laravel Sail

Para montar un entorno de desarrollo usaremos Laravel Sail, que facilita la configuración con Docker.

5.1 1. Crear un Proyecto con Laravel 11 usando Sail

Ejecuta el siguiente comando para crear un nuevo proyecto utilizando Sail:

```
curl -s "https://laravel.build/laravel11-api?with=pgsql,redis" | bash
```

Por defecto, el stack instalado incluye MySQL, Redis, Meilisearch, Mailhog y Selenium, pero puedes personalizarlo.

Inicia la aplicación con:

```
cd laravel11-api && ./vendor/bin/sail up -d
```

Opcionalmente, define un alias para evitar escribir repetidamente `vendor/bin/sail`:

```
alias sail='bash vendor/bin/sail'
```

6 Instalación de Sanctum y configuración de autenticación

Laravel 11 no incluye el archivo `routes/api.php` por defecto, y tampoco Sanctum. Para instalar el scaffolding de la API, ejecuta:

```
sail php artisan install:api
```

Este comando instala Sanctum para autenticación con tokens, crea el archivo `routes/api.php` y ejecuta las migraciones necesarias.

Si necesitas instalar Sanctum manualmente:

```
sail composer require laravel/sanctum
sail artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
sail artisan migrate
```

En el modelo `User.php`, asegúrate de incluir el trait `HasApiTokens`:

```
namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;
}
```

Para proteger las rutas de la API, usa el middleware `auth:sanctum` en `routes/api.php`:

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\V1\AuthController;

Route::prefix('auth')->group(function () {
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/logout', [AuthController::class, 'logout'])->middleware('auth:sanctum');
    Route::get('/user', [AuthController::class, 'user'])->middleware('auth:sanctum');
});
```

Crea el controlador `AuthController.php` con:

```
sail artisan make:controller Api/V1/AuthController
```

Y añade la lógica de autenticación:

```
namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use App\Models\User;

class AuthController extends Controller
{
    public function login(Request $request)
    {
        $credentials = $request->validate([
```

```

        'email' => 'required|email',
        'password' => 'required',
    ]);

    if (!Auth::attempt($credentials)) {
        return response()->json(['message' => 'Credenciales inválidas'], 401)
    }

    $user = Auth::user();
    $token = $user->createToken('auth_token')->plainTextToken;

    return response()->json(['access_token' => $token, 'token_type' => 'Bearer']);
}

public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();
    return response()->json(['message' => 'Sesión cerrada correctamente']);
}

public function user(Request $request)
{
    return response()->json($request->user());
}
}

```

7 CRUD para Posts

7.1 1. Crear modelo y controlador

```
sail artisan make:model Post -m -c --api
```

Esto genera:

- **Modelo:** app/Models/Post.php
- **Migración:** database/migrations/...create_posts_table.php
- **Controlador:** app/Http/Controllers/Api/V1/PostController.php

Modifica la migración para definir la tabla posts:

```

public function up()
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->string('title');
        $table->string('image')->nullable();
    });
}

```

```
        $table->text('description');
        $table->timestamps();
    });
}
```

Aplica las migraciones:

```
sail artisan migrate
```

7.2 2. Definir rutas de la API

```
use App\Http\Controllers\Api\V1\PostController;

Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('posts', PostController::class);
});
```

7.3 3. Implementar métodos en PostController.php

```
namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Models\Post;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Validator;

class PostController extends Controller
{
    public function index()
    {
        return Post::latest()->paginate();
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|max:255',
            'description' => 'required',
            'image' => 'image|max:1024',
        ]);

        $validated['user_id'] = Auth::id();
        $post = Post::create($validated);

        return response()->json($post, 201);
    }
}
```

```
public function destroy(Post $post)
{
    $post->delete();
    return response()->json(['message' => 'Post eliminado'], 200);
}
}
```

8 Conclusión

Laravel 11 ha simplificado la estructura de proyectos, eliminando configuraciones predefinidas. Con esta guía, hemos adaptado el sistema de autenticación con Sanctum y configurado un CRUD de posts protegido por autenticación.